PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE – ACTIVIDADES
ESTRUCTURAR LA PROPUESTA TÉCNICA

Actividad

Proyecto-Express

Presentado a: Yoly Alexandra Guevara Muños
Por Aprendiz: Jenny Carolina Marrugo Ussa
Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 20 de octubre del año 2026

Introducción

En este documento se muestra el desarrollo de una API básica utilizando Node.js y Express, creada como parte de las actividades del programa ADSO del SENA. El propósito principal es aplicar los conceptos básicos del backend, como la creación de rutas GET y POST, el uso de middlewares para procesar JSON y registrar peticiones, y la realización de pruebas funcionales para verificar las respuestas del servidor.

¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta? Es una función que se ejecuta en el medio entre que llega una petición del cliente y el servidor responde. Sirve para procesar datos, validar o revisar cosas antes de que la ruta principal actúe.

¿Qué significan los objetos req y res? req (request) es lo que manda el cliente (como datos), y res (response) es lo que el servidor le devuelve al cliente.

¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta específica y uno de manejo de errores? El global se aplica a todas las rutas de la aplicación; el de ruta específica solo se activa cuando se visita una URL concreta; y el de manejo de errores se encarga específicamente de atrapar y responder cuando ocurre un fallo en el servidor.

¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía JSON? Convierte los datos JSON que manda el cliente en un objeto de JavaScript legible. Si no se pone, req.body se cometeria error

¿Por qué importa el orden en que se registran los middleware y las rutas? Porque Express lee el código de arriba hacia abajo de forma secuencial. Si pones una ruta antes de un middleware necesario (como express.json()), la ruta no alcanzará a procesar los datos correctamente.

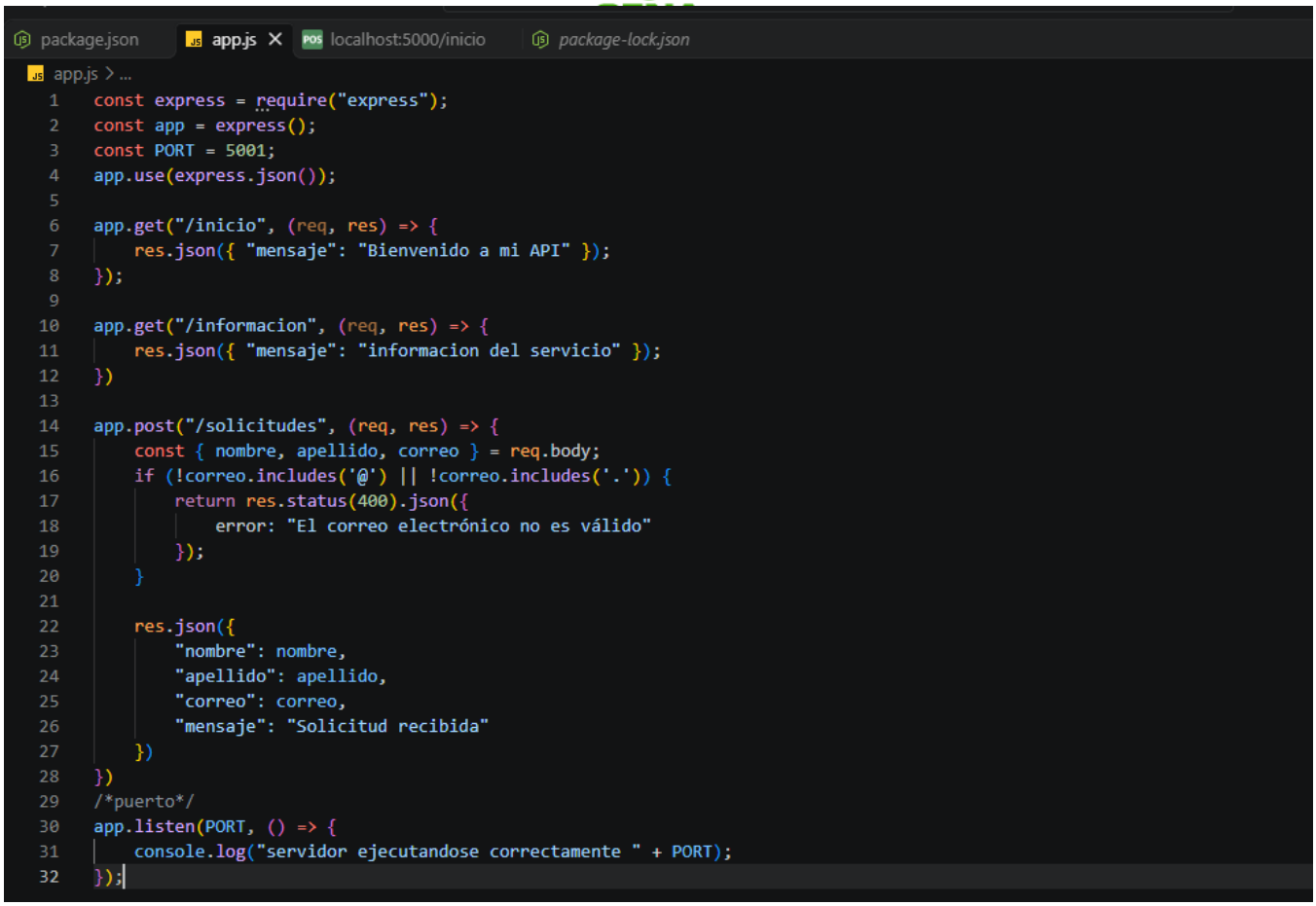
¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo? La petición se quedará congelada cargando indefinidamente y el cliente nunca recibirá una respuesta del servidor.

¿Cuál es la diferencia entre un middleware y la función que atiende una ruta? El middleware procesa o revisa la petición de forma general es el que valida o registra los datos, mientras que la función de la ruta es la encargada final de devolver la respuesta específica para esa dirección.

¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite? Que cualquier usuario podría enviar datos vacíos, incorrectos o malintencionados, lo que podría causar errores en el sistema o fallos de seguridad.

Incluye un ejemplo cotidiano, explicado sin código, que ayude a entender la función de un middleware. Un vigilante en la entrada de un edificio: revisa quién entra y registra los datos antes de dejar pasar a la persona a las oficinas principales.

5. Construcción del middleware propio



¿Dónde se ubicó el middleware propio y por qué se eligió esa ubicación?
Se ubicó justo después de express.json() y antes de las rutas. Se puso ahí para que se ejecute en cada petición que llegue antes de que el servidor empiece a procesar las rutas.

Comprobación en rutas GET e información: Al consultar rutas como /inicio o /informacion en Thunder Client, la consola muestra de inmediato el mensaje con el método y la ruta.

Comprobación en peticiones POST: Al enviar datos a /solicitudes, la consola también registra el método POST y la ruta utilizada.

¿Qué cambia si el middleware se coloca después de las rutas en vez de antes?
Si se pone al final, las peticiones que ya pasaron por las rutas no alcanzarán a ser registradas por el middleware, o se quedarán bloqueadas si no tienen un next().
¿Cómo se transmite el control al siguiente middleware o a la ruta? Utilizando la función next(), que le avisa a Express que el middleware terminó su tarea y puede continuar con el siguiente paso.

6. Códigos de estado HTTP y manejo de errores

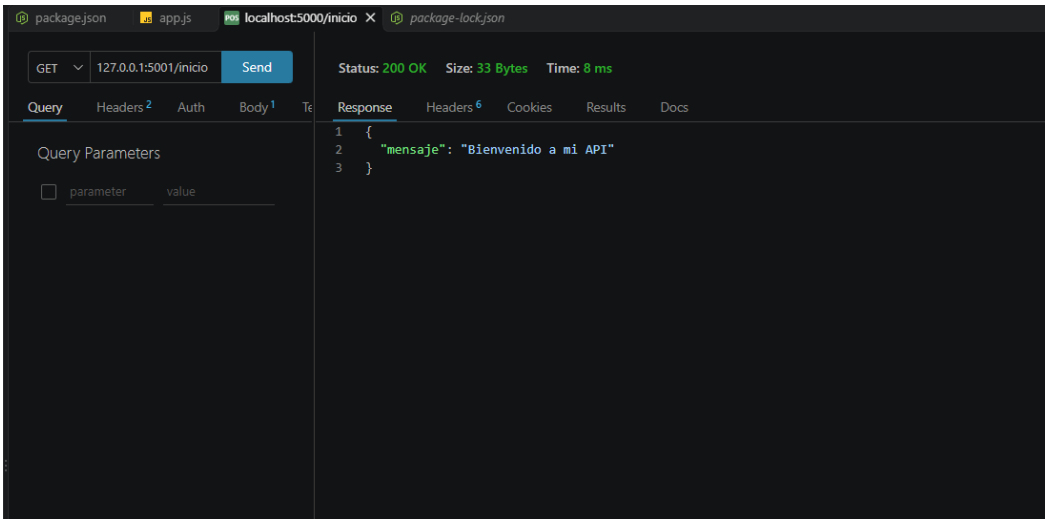
- **200 OK**: Se usa cuando una petición se procesa con éxito total, por ejemplo, al consultar las rutas GET /inicio o GET /informacion.
- **201 Created**: Indica que la petición se procesó bien y que se creó un nuevo registro o solicitud, ideal para cuando se envían datos con POST /solicitudes.
- **400 Bad Request**: Se usa cuando el cliente envía datos incompletos, erróneos o faltantes, por

lo que el servidor no puede procesarlos.

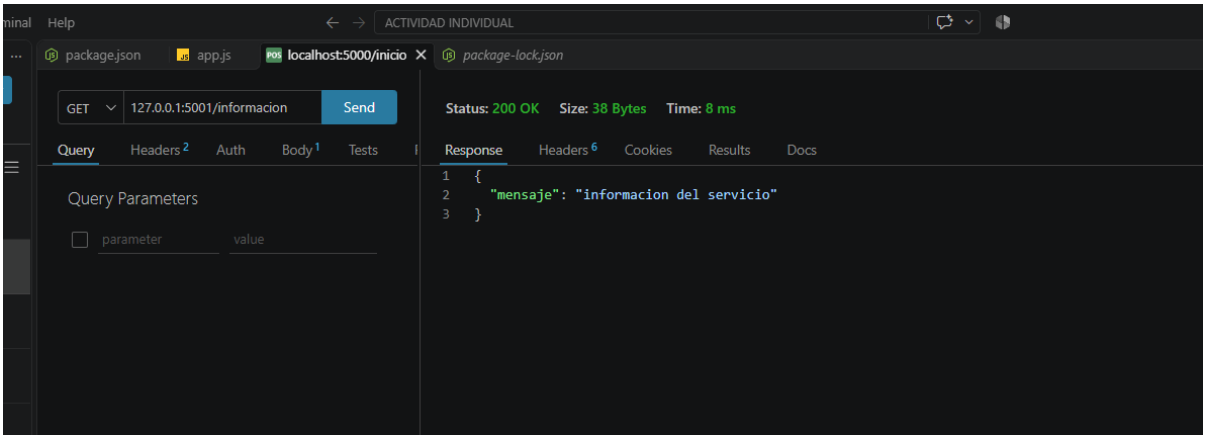
- **404 Not Found**: Significa que la ruta o dirección que está buscando el cliente no existe en el servidor.
- **500 Internal Server Error**: Ocurre cuando hay un fallo inesperado del lado del servidor (un error de programación o base de datos que no se controló)

Pruebas de la API

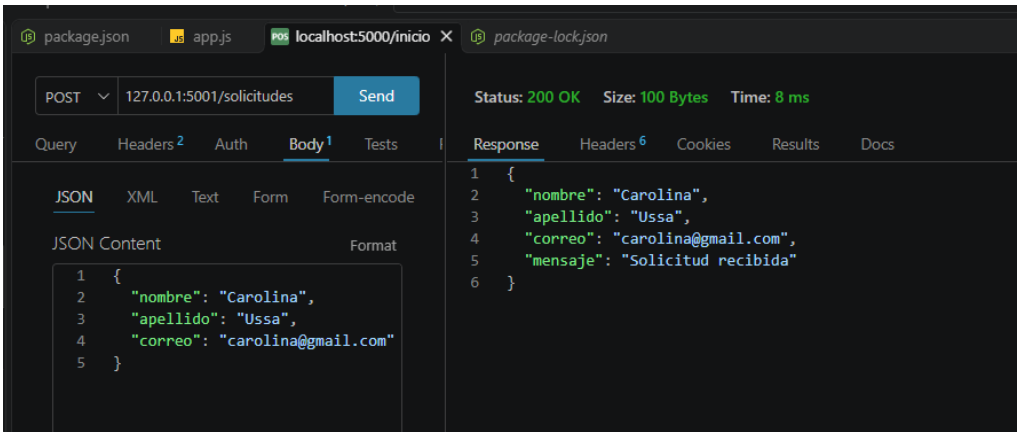
- **GET /inicio**: Se prueba consultando la ruta y se verifica que devuelve la respuesta con el mensaje de bienvenida en formato JSON y un estado exitoso.



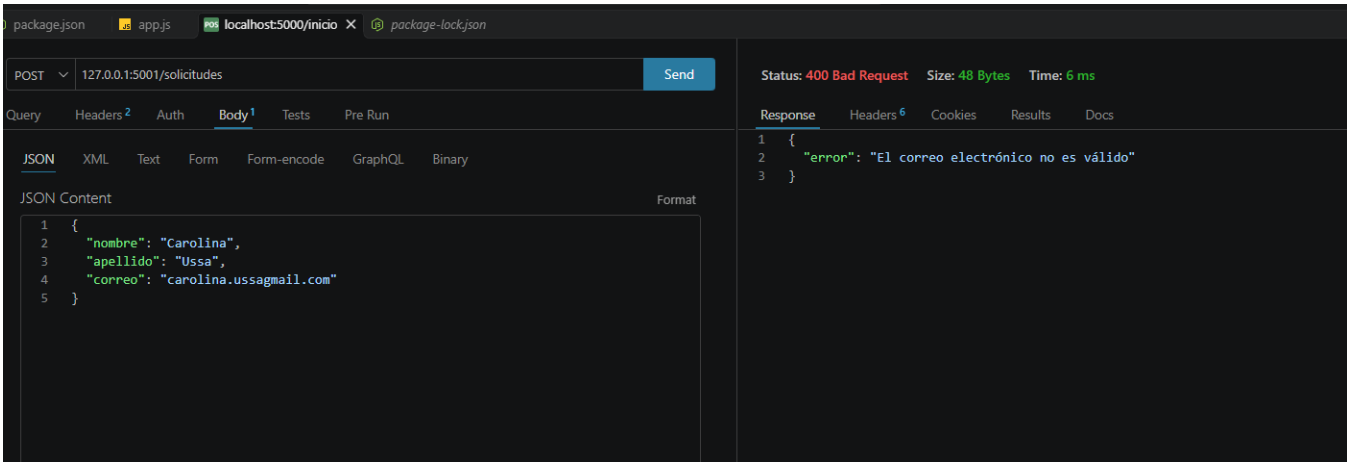
- **GET /informacion**: Se comprueba que al ingresar a esta ruta el servidor responde correctamente con la información general del servicio.



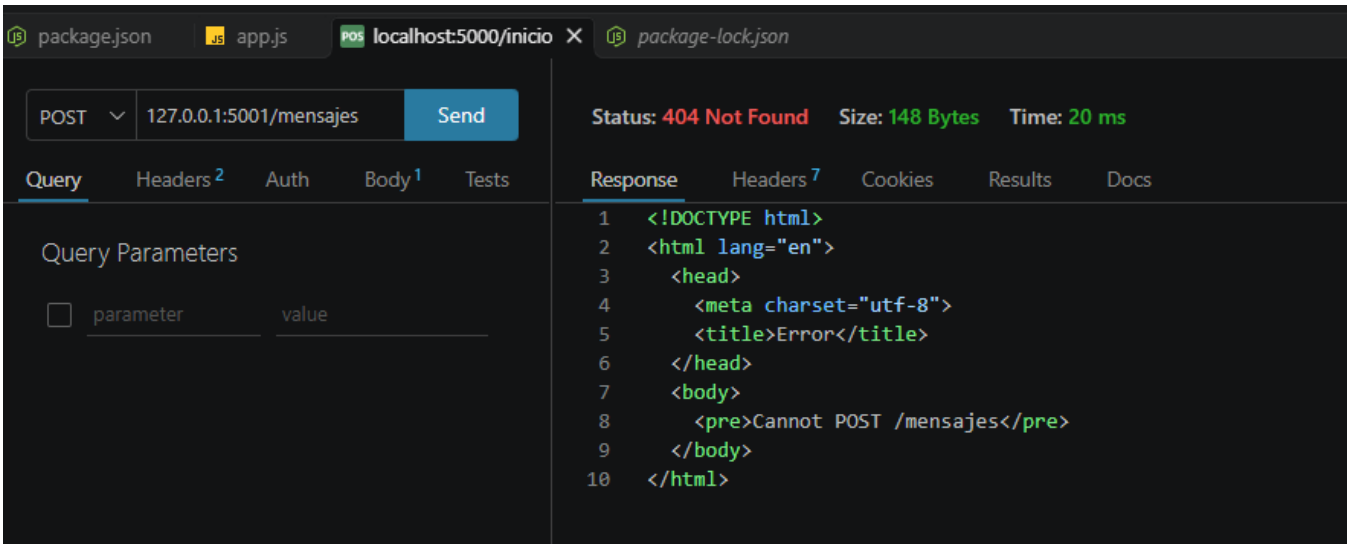
- **POST /solicitudes (Datos válidos)**: Se envía un cuerpo en formato JSON con el nombre, apellido y correo, verificando que la API responda confirmando que la solicitud fue recibida.



POST /solicitudes con correo de formato inválido: Se envía una petición con los datos, pero modificando el correo (por ejemplo, sin incluir el símbolo @ o el punto), y se verifica que la API responda indicando el error en los datos.



Consultar una ruta que no exista y verificar la respuesta 404: Al escribir una URL incorrecta en el cliente, se comprueba que el servidor responde con el código 404 Not Found informando que la ruta no existe



Verificar en la consola que el middleware propio registre las peticiones: Se revisa la terminal de Visual Studio Code para confirmar que cada vez que se hace una petición (GET o POST), aparece impreso automáticamente el método y la ruta consultada

Probar qué ocurre si se envía un cuerpo JSON y explicar el papel de express.json(): Se envía información en formato JSON desde el cliente; gracias a express.json(), el servidor puede leer esos datos en el req.body sin problemas. Si se quitara esta línea, el servidor no podría interpretarlos y arrojaría un error

8. Documento técnico obligatorio: paso a paso

- **Instalación de Node.js y npm**

Por qué se realiza: Permite ejecutar JavaScript fuera del navegador y administrar los paquetes del proyecto.
Qué ocurriría si se omite: No se podría correr ningún servidor ni instalar Express.

- **Creación de la carpeta y apertura del proyecto**

Por qué se realiza: Para mantener ordenados los archivos de trabajo en el computador.
Qué ocurriría si se omite: Habría desorden y los archivos quedarían dispersos en el escritorio.

- **Inicialización del proyecto y package.json**

Por qué se realiza: Crea el archivo de configuración que guarda las dependencias y scripts del proyecto.
Qué ocurriría si se omite: Node no sabría qué librerías usa el programa ni cómo ejecutarlo.

- **Instalación de Express**

Por qué se realiza: Facilita la creación del servidor, las rutas y la gestión de peticiones web.
Qué ocurriría si se omite: Tocaría programar todo el protocolo HTTP desde cero con código puro de Node, lo cual es muy difícil.

- **Instalación y configuración de Nodemon**

Por qué se realiza: Reinicia el servidor automáticamente al guardar cambios en el código.
Qué ocurriría si se omite: Tocaría apagar y prender el servidor manualmente con node app.js cada vez que modifiquemos algo.

- **Archivo principal y puerto**

Por qué se realiza: Es donde arranca la aplicación y define el canal (puerto) por donde el servidor escucha las peticiones.
Qué ocurriría si se omite: La aplicación no tendría punto de inicio ni sabría por dónde recibir conexiones.

- **Middleware para analizar JSON (express.json())**

Por qué se realiza: Prepara y transforma los datos enviados por el cliente para que se puedan leer en el req.body.
Qué ocurriría si se omite: El servidor no entendería los datos enviados en formato JSON y arrojaría error al intentar leerlos.

- **Middleware propio**

Por qué se realiza: Registra en la consola de forma automática cada método y ruta que se consulta.
Qué ocurriría si se omite: No tendríamos un registro en consola de las peticiones que van llegando.

- **Rutas GET y POST**

Por qué se realiza: Permiten consultar información y recibir datos enviados por el usuario.
Qué ocurriría si se omite: La API no tendría páginas ni endpoints disponibles para responderle al cliente.

- **Git y repositorio remoto**

Por qué se realiza: Permite llevar el control de versiones y respaldar el código en GitHub.
Qué ocurriría si se omite: No habría respaldo del trabajo y se perdería el historial de cambios.

Conclusiones

El desarrollo de este taller permitió comprender de forma práctica cómo funciona una API utilizando Node.js y Express, facilitando la creación de rutas GET y POST de manera ordenada. Se entendió la importancia vital de los middlewares, especialmente express.json(), ya que sin él el servidor no podría interpretar los datos enviados por el usuario en formato JSON.
El uso de Nodemon optimizó el proceso de desarrollo al reiniciar el servidor automáticamente tras cada cambio, y las herramientas de pruebas permitieron verificar las respuestas y los registros en consola de forma exitosa.